1) We will use the chat name “Cmd Rev A”.
2) I already defined command structure and its reply from remote system. It is working fine and didn’t have any problem. I like your help to extend the fixed concept to dynamically by using methods, dictionaries and other suitable properties of python such a way that addition of new command will be very easy, does not need multiple calculations etc similar to example string, all the transmit parameters, received parameters etc become handy to use.
3) There will be some look tables, where I need to add names, size etc and call prepare frame, send and receive the reply. The whole process gets executed with data, message, warning etc.
4) The **command and its reply frames** have two parts, called header and data. The header will be first and data will next one as shown in table below.

Header	Data

5) The header is divided in multiple sections as follows and having fix length of 10 bytes. The version is fixed constant value. The data format is in big endian. The CRC is calculated for header only, its type is CRC-CCITT (0xFFFF).

Version	Service type	Data length (header excluded)	Remote Serial Number	Header CRC
one byte	one byte	two bytes	four bytes	two bytes
0	1	2,3	4,5,6,7	8,9
0x15	0x00	0x0A, 0x00	0xD2, 0x04, 0x00, 0x00	0x9D, 0x5B

Version = 0x15

Service type = 0x00

Below six types of services are defined now. This value used by sender, did he receive ack or nak to take call for his request, what did remote do with it.

```
Service type = {
   CMD_CODE_ACK    = 0x00,
   CMD_CODE_NAK    = 0x01,
   CMD_CODE_RPLY = 0x02,
   CMD_CODE_SET    = 0x03,
   CMD_CODE_GET    = 0x04,
   CMD_CODE_DATA = 0x05,
};
```

Data length without header = 0x000A

Remote Serial Number = 0x000004D2

Header CRC = 0x5B9D

6) The data section of frame is divided in multiple sub-sections as follows and its length is given in header at location marked “Data length without header”, i.e., 3rd section of header of frame. The definition of data sections is given below.

Module Name	Module’s Parameter Name	Parameter value
3 ASCII chars	Variable length of ASCII chars	Variable length: ASCII or number in little endian
ADC	SRATE	0x00, 0x80

```
Module Name = {
   "ADC",              // ADC commands
   "SYS",              // System commands
```

"USD", // uSD card commands
"USB", // USB Pen commands
"FFS", // File System commands
};
ADC Module's Parameters
ADC_Parameters = {
"SRATE", // ADC Sampling Rate
"EVNT_LVL", // ADC event detection level in counts
"BITVOL", // ADC Least Count value in uV
"NOTCH", // Notch Filter On or Off
"NOTCHHZ", // Notch Filter Frequency
"CHANNEL", // Enable number of channels
"GEOHZ", // Geophone HPF Hz
"REXTD", // Geophone Range Extend yes/no
"REXTDHZ", // Geophone Range Extend Hz
"CAL", // Analog Channel calibration
};
ADC Module's SRATE Parameter value = 0x0800

As above example of ADC module, there will be details for SYS, USD, USB and FFS. Once I know placeholders, I can fill the same and extend the whole mechanism further.

7) There are two types of command frames
   a) set_data - will send frame of data with service type (= CMD_CODE_SET) to remote. If message received is correct, remote system send reply by changing service type either CMD_CODE_ACK or CMD_CODE_NAK with length trimmed without module parameter value size, (i.e., Data length - Module's Parameter value size in bytes).
   b) get_data - will send frame of data with service type (= CMD_CODE_GET) to remote. If message received is correct, remote system send reply by changing service type either CMD_CODE_ACK or CMD_CODE_NAK with length increased by module parameter value size, (i.e., Data length + Module's Parameter value size in bytes).
8) Using above data for ADC module and sampling rate to get from remote, the transmit and receive frames will be like this.
   a) Send_frame = Version, CMD_CODE_GET, 0x08, 0x00, 0x1B, 0x11, "ADC", "SRATE"
   b) Rcv_ok_frame = Version, CMD_CODE_ACK, 0x0A, 0x00, 0x9D, 0x5B, "ADC", "SRATE", 0x00, 0x80
   c) Rcv not ok frame = Version, CMD_CODE_NCK, 0x08, 0x00, 0xBC, 0x68, "ADC", "SRATE"

   The send frame with above data and send size will be len(Send_frame), expected return frame be len(Send_frame) + size of parameter, here 2 bytes more. In case of failure, we will get same as transmitted. This is catch, how to handle it. One option, even it is failing case, still size of ok and not ok frame can be made to same so handling will be easy in Python. Python has now possibility, pass, fail or time out if remote is dead or not available. Dead case, we can't open port, so it can be handled at top level. In current implementation, all are hard coded values and system is working, now we want to go dynamically, and all are taken from lookup tables to make flexible system.

   Does this make sense to you. We can have one file called command.py which has all these things

to isolate from others and make modular and structure system.

We will analysis the whole things and debate on it and once all the points are clear, will create python program.